

8

Sensores en MakeCode — el micro:bit que siente

Guía 14-8-TIC

LO QUE TE LLEVAS HOY

Programa en MakeCode que use al menos 2 sensores del micro:bit (luz, temperatura, botones A/B, acelerómetro o movimiento) para disparar 1 acción visible (mostrar ícono, número, sonido o mensaje) + bitácora de pruebas con al menos 5 escenarios distintos donde se documente qué entrada se dio, qué respuesta esperabas y qué respuesta dio el micro:bit + 1 ajuste hecho al programa después de las pruebas

ESTA GUÍA ES DE

Nombre completo

Grupo

Fecha

Autor: Dr. Álvaro Cárdenas Orozco

Metodología MILC · apertura ancestral · pensamiento computacional · triángulo Dussel–estoicismo–Floridi

Apertura: Sensores en MakeCode — el micro:bit que siente

Saber ancestral

En las fincas del Valle del Cauca, en las casas con patio del campo y en los corregimientos de la zona cafetera, había un compañero del oficio que dormía menos que el dueño: **el perro guardián**. El perro no tenía instrucciones escritas ni reloj, pero ejercía una función específica: **sentir lo que el dueño no siente**. Olía al extraño cuando todavía estaba a 100 metros del portal. Escuchaba el ruido del zorro antes de que llegara al gallinero. Sentía el motor de la motocicleta en el camino antes de que se viera la luz. Y cuando percibía algo, no actuaba solo: **avisaba** con su ladrido, su gemido o su carrera al patio principal. El dueño, al escuchar el aviso, decidía qué hacer (asomarse, buscar el machete, encender la luz). El perro no decidía; **percibía y avisaba**. Esa función ancestral del perro guardián tiene un nombre en computación física: **sensor**. Un sensor digital es exactamente eso: **algo que percibe el mundo y le avisa al programa para que decida qué hacer**.

Saber contemporáneo

El **micro:bit** es una placa de computadora pequeña, del tamaño de una tarjeta de crédito, diseñada para enseñar programación con sensores físicos. Incluye varios **sensores integrados**: (1) **Botones A y B**: detectan pulsación humana. (2) **Sensor de luz**: detecta el nivel de luz ambiente (0 oscuro, 255 luz fuerte). (3) **Sensor de temperatura**: detecta la temperatura aproximada del chip (cerca de la ambiente). (4) **Acelerómetro**: detecta movimiento y orientación (agitar, inclinar, caída). (5) **Brújula**: detecta dirección magnética. **MakeCode** (makecode.microbit.org) es el editor visual donde se programa con bloques de colores y se puede simular el código antes de cargarlo al micro:bit físico. La regla profesional del oficio: **los sensores leen el mundo, el código decide la respuesta, los actuadores la ejecutan**. Hoy trabajas la primera parte: usar 2 sensores para que el micro:bit *sienta* y dispare una acción visible.

Pregunta-puente

¿Qué sabía el perro guardián de la finca al percibir antes y avisar al dueño, que el programador novato olvida cuando trata el sensor como si fuera un cálculo más? ¿Y por qué el micro:bit con 2 sensores combinados puede tomar decisiones más útiles que con uno solo?

Saber y saber hacer

Al terminar podrás: (1) **identificar** qué sensor del micro:bit es apropiado para detectar una entrada concreta del mundo; (2) **aplicar** los bloques de MakeCode para leer al menos 2 sensores y disparar una acción condicional; (3) **crear** un programa funcional que combine 2 sensores con lógica clara; (4) **evaluar** el programa probándolo en 5 escenarios distintos y haciendo al menos 1 ajuste documentado.

Autor	Dr. Álvaro Cárdenas Orozco
DBA	Pensamiento computacional aplicado a sistemas de control con sensores y actuadores (MEN, T&I)
Referentes	Pensamiento computacional · MakeCode / micro:bit · Logos como razón universal
Producto final	Programa en MakeCode que use al menos 2 sensores del micro:bit (luz, temperatura, botones A/B, acelerómetro o movimiento) para disparar 1 acción visible (mostrar ícono, número, sonido o mensaje) + bitácora de pruebas con al menos 5 escenarios distintos donde se documente qué entrada se dio, qué respuesta esperabas y qué respuesta dio el micro:bit + 1 ajuste hecho al programa después de las pruebas

→ Antes de empezar, mira el viaje completo. En las sesiones 1-3 aprendiste a pensar lógica y a escribir algoritmos en papel. Hoy haces el salto al **código real** con sensores físicos. Las próximas sesiones (actuadores S5, umbrales S6, depuración S7) construyen sobre la habilidad de leer sensores que aprendes hoy.

Ruta MILC: así vamos a aprender

1

Escucha
Observo, escucho
y pregunto.

2

Entender
Sistematizo
y ordeno.

3

Hacer
Construyo y aplico.

4

Cierre
Evalúo y reflexiono.

→ Antes de teorizar, vas a abrir MakeCode (makecode.microbit.org) y explorar el simulador: el micro:bit virtual que aparece en pantalla. Vas a tocar sus botones, agitarlo con el ratón, cambiar la luz, ver qué pasa. Es exploración antes de la programación.

Estación 1 de 3 · Escucha

Escena para escuchar

Actividad 1 · IDENTIFICA — Exploración del simulador (15 min · individual). Abre makecode.microbit.org y elige *Nuevo proyecto*. Mira el simulador del micro:bit (a la izquierda) y la paleta de bloques (a la derecha). Sin programar todavía, explora: (1) los botones A y B en el simulador (¿se pueden hacer clic?). (2) el sensor de luz (busca el control con un sol pequeño). (3) el acelerómetro (busca el control con flecha). (4) la pantalla de LEDs 5×5. Anota: ¿qué sensores tiene el micro:bit?, ¿qué puede mostrar?, ¿qué te llama la atención?.

MARCA CUANDO LO HAYAS HECHO

- ☐ Abrí MakeCode y vi el simulador.
- ☐ Identifiqué los sensores disponibles.
- ☐ Vi la pantalla de LEDs como salida principal.

Lo que va al cuaderno (Actividad 1 · IDENTIFICA). Título: «Actividad 1 — Exploración micro:bit». **Formato:** dibujo o esquema del micro:bit con sus partes etiquetadas + lista de sensores y salidas. **Sabes que terminaste cuando** conoces qué puede percibir y qué puede mostrar el micro:bit.

→ Ya conoces el simulador. Ahora vas a entender la **anatomía de los sensores**: los 5 sensores principales del micro:bit, los bloques de MakeCode que los leen, los 5 errores típicos del primer programa con sensores.

Estación 2 de 3 · Entender

Lo que vas a entender

El simulador te enseñó qué hay. Ahora necesitas la **anatomía de la programación con sensores**: cómo se leen, qué bloques se usan, cómo se combinan con lógica condicional.

- 1 Los **5 sensores principales** del micro:bit y sus bloques se descomponen así: (1) **Botones A/B**: bloque “cuando se presiona el botón A” (categoría Entrada). Evento que dispara código. (2) **Sensor de luz**: “nivel de luz” (categoría Entrada). Devuelve número 0-255. Útil para detectar oscuridad. (3) **Sensor de temperatura**: “temperatura (°C)” (Entrada). Devuelve número aproximado. (4) **Acelerómetro**: bloque “cuando se agita” (eventos) o “aceleración (g) X/Y/Z” (valores continuos). (5) **Brújula**: “orientación de la brújula” (Entrada).
- 2 Los **bloques de salida** (acciones visibles) son: (a) “mostrar ícono”: muestra un dibujo predefinido en la pantalla LED. (b) “mostrar número”: muestra cifras desplazándose. (c) “mostrar cadena”: muestra texto. (d) “tocar nota”: emite sonido (requiere altavoz). (e) “encender LED en X,Y”: control individual de los 25 LEDs.
- 3 La **estructura básica** de un programa con sensores tiene 2 partes: (a) **al iniciar**: código que se ejecuta una sola vez al encender el micro:bit (configuraciones, mensaje de bienvenida). (b) **para siempre**: código que se repite continuamente (lectura de sensores, decisiones, acciones). Dentro del *para siempre* se ubican los if-else que combinan sensores: si nivel de luz < 50 y se presiona botón A entonces mostrar ícono triste.
- 4 Algoritmo de construcción: (1) define qué hace el programa en 1 frase. (2) identifica los 2 sensores que usarás. (3) define la acción de salida. (4) escribe el if-else en pseudocódigo. (5) traduce a bloques de MakeCode. (6) prueba en el simulador. (7) carga al micro:bit físico si está disponible. (8) prueba en al menos 5 escenarios.

Anatomía del programa con sensores

Al iniciar: configuración inicial.

Para siempre: bucle principal.

Sensores: botones / luz / temperatura / acelerómetro / brújula.

Salidas: ícono / número / sonido / LED individual.

Combinación: 2 sensores + 1 acción con if-else.

Errores comunes y su corrección

(1) **Sensor sin condición:** leer “nivel de luz” pero no comparar con un umbral. (2) **Acción sin disparador:** “mostrar ícono” fuera de un evento o bucle. (3) **Olvidar el para siempre:** el programa se ejecuta una sola vez y no responde a cambios. (4) **No probar en simulador antes de cargar:** pierdes tiempo si hay error. (5) **Mezclar bloques sin lógica:** arrastrar bloques al azar sin saber qué hace cada uno.

Lo que va al cuaderno (Actividad 2 · APLICA). Título: «Actividad 2 — Anatomía del programa».

Formato: (a) los 5 sensores con su bloque; (b) las salidas; (c) estructura básica (al iniciar + para siempre); (d) los 5 errores con marca. **Sabes que terminaste cuando** podrías escribir un programa simple sin abrir esta guía.

→ Tienes el método. Toca aplicarlo: programa que use al menos 2 sensores (botón + luz, o acelerómetro + temperatura, o botones A y B combinados) y dispare 1 acción visible. Probar en 5 escenarios y ajustar.

Estación 3 de 3 · Hacer

Cómo vas a construir tu producto

Actividad 3 · CREA + EVALÚA — Programa con 2 sensores + 5 pruebas (40 min · individual o parejas). Hora de crear. Programa que use 2 sensores combinados con lógica y dispare 1 acción visible. Pruébalo en 5 escenarios y documenta lo que pasa en cada uno.

- 1 Ideas concretas para combinar 2 sensores: (1) **Termómetro inteligente:** si temperatura >25 muestra ícono caliente, si <18 muestra frío, intermedio muestra OK. Botón A muestra la cifra exacta. (2) **Vela digital:** detecta oscuridad con sensor de luz, muestra LEDs encendidos; al agitar (acelerómetro) los apaga “soplando”. (3) **Cara expresiva:** botón A muestra cara feliz, botón B cara triste, agitar muestra cara confundida. (4) **Detector de movimiento:** si está oscuro Y se agita, suena alarma; si solo está oscuro, muestra punto; si solo se agita en luz, ignora.

- 2 Sigue el patrón “**percibir** → **decidir** → **actuar**”: el programa lee los sensores, combina con if-else (de la sesión 1), dispara la acción. Esa secuencia es la phronesis del oficio: nunca actuar sin percibir, nunca decidir sin condición clara.
- 3 La bitácora de 5 pruebas tiene estructura mínima: (a) **Escenario**: combinación de sensores (luz alta + botón A presionado). (b) **Respuesta esperada**: qué creías que iba a mostrar. (c) **Respuesta real**: qué mostró efectivamente. (d) **Coincide o no**. Después de los 5 escenarios, identifica 1 ajuste a hacer (cambiar el umbral, cambiar el ícono, agregar otra condición) e implementarlo.
- 4 Algoritmo: (1) elegir idea. (2) escribir pseudocódigo de la lógica. (3) construir en MakeCode con bloques. (4) probar en simulador. (5) hacer 5 pruebas variando entradas. (6) llenar bitácora. (7) identificar 1 ajuste. (8) aplicar el ajuste. (9) verificar mejora.

Checklist antes de entregar

- ☐ 2 sensores usados en el programa.
- ☐ 1 acción de salida visible.
- ☐ Lógica con if-else combinada.
- ☐ Probado en simulador.
- ☐ 5 escenarios documentados en bitácora.
- ☐ Al menos 1 ajuste hecho después de pruebas.
- ☐ Código compartido (link de MakeCode o archivo .hex).

Plantilla de trabajo

Idea del programa. *Una frase clara.*

Sensores usados. *Cuáles 2 y por qué.*

Acción de salida. *Qué muestra/sonido emite.*

Pseudocódigo. *La lógica antes de bloques.*

Pruebas. *5 escenarios con esperado/real.*

Ajuste. *1 cambio aplicado.*

Lo que va al cuaderno (Actividad 3 · CREA + EVALÚA). Título: «Actividad 3 — Mi primer programa con sensores». **Formato:** (a) idea + sensores + acción; (b) pseudocódigo; (c) tabla 5 filas de bitácora; (d) descripción del ajuste hecho. **Sabes que terminaste cuando** el programa responde como esperas en al menos 4 de los 5 escenarios.

→ Lo que sigue resume qué entregas hoy: programa funcional + bitácora de 5 pruebas + 1 ajuste documentado. El criterio principal: que el programa responda como esperas en los 5 escenarios, o que la bitácora explique honestamente cuándo no lo hace.

Producto de la sesión

Programa con 2 sensores + 5 pruebas documentadas + 1 ajuste.

Criterios de calidad

(1) Programa funcional con 2 sensores. (2) Lógica con if-else combinada. (3) Probado en simulador. (4) Bitácora de 5 escenarios. (5) 1 ajuste documentado. (6) Código compartible (link o archivo).

→ *Antes de cerrar, mira el programa desde las cinco dimensiones humanas. Probar con varios escenarios antes de declarar funcional es respeto por el oficio; declarar funcional con una sola prueba es engaño con apariencia técnica.*

Evaluación liberadora · cinco dimensiones

Sentido de la evaluación

Evaluar no es solo revisar una nota. Es reconocer qué aprendí, qué cambió en mí, qué emociones manejé, cómo me relaciono con la ciudad y la comunidad, y qué le dejaré a quien viene detrás.

Mi reflexión liberadora en cinco dimensiones. Completa cada frase iniciada:

Dimensión	Mi respuesta (frame starter)	Algo que descubrí
1. Personal	Lo que más cambió en mí fue _____	_____
2. Emocional	La emoción más fuerte fue _____ y la regulé _____	_____
3. Ciudadana	En democracia esto importa porque _____	_____
4. Local (Cartago)	En mi barrio sería útil para _____	_____
5. Intergeneracional	A mi abuela/abuelo le diría _____	_____

Para la próxima sustentación. Vuelve a esta tabla en seis meses. Verás que las respuestas cambian — no porque lo aprendido cambie, sino porque tú cambias. La evaluación liberadora es una conversación contigo a través del tiempo.

→ *Y para terminar, tres voces sobre los sistemas que perciben. Dussel sobre los sensores que cuidan o vigilan, Marco Aurelio sobre la prueba que verifica, Floridi sobre los sistemas que aumentan la inteligencia humana.*

Triángulo de pensamiento · cierre filosófico

Dussel · lente del nosotros

Un sensor que cuida es liberador; un sensor que vigila sin consentimiento es opresor.

Los sensores pueden tener dos usos: cuidar (alertar cuando hay peligro, ayudar a quien lo necesita) o vigilar (controlar sin consentimiento, registrar sin avisar). La filosofía de la liberación pide elegir siempre el primero. Cuando programas un detector con micro:bit, piensa: *¿este dispositivo cuida al usuario o lo vigila?*. Esa pregunta ética abre el oficio antes de la técnica.

Mi programa con sensores, ¿cuida o vigila? ¿El usuario sabría qué hace este micro:bit si lo encontrara funcionando?

Estoicismo · Marco Aurelio · cuidado interior

Probar con varios escenarios es virtud; declarar funcional con una sola prueba es vanidad.

Es tentador declarar el programa funcional después de la primera vez que el simulador hace lo esperado. La disciplina estoica recomienda lo opuesto: *prueba con 5 escenarios distintos antes de confiar*. La bitácora de pruebas es la forma estoica de respetar la realidad del código: no asumir que funciona; verificar que funciona.

¿Estoy probando con varios escenarios o me convencí con una sola prueba?

Floridi · lente de la infoesfera

Los sistemas que aumentan la inteligencia humana sin reemplazarla son la nueva ética del oficio digital.

Un buen sensor con código no pretende sustituir al humano: lo asiste. El termómetro inteligente no decide por ti si abrigarte; te avisa el estado y tú decides. La ética digital contemporánea valora esa colaboración entre humano y máquina: *el sistema percibe, el humano decide*. Tu programa de hoy te entrena en esa práctica.

¿Mi programa asiste al usuario o intenta decidir por él?

Nota para el docente. Las tres formulaciones del triángulo son la síntesis del autor de la guía, no citas textuales de los pensadores. Si vas a citarlos en clase, remite a la obra original.

Mi autoevaluación · marca una casilla por fila

Criterio	Lo logré	En proceso	Necesito apoyo
Comprensión del tema	☐ Explico las ideas con mis palabras.	☐ Reconozco las ideas, falta precisión.	☐ Necesito repasar lo básico.
Pensamiento computacional	☐ Usé los pilares al armar mi producto.	☐ Usé algunos pilares.	☐ Necesito releer la estación 2.
Aplicación y producto	☐ Mi producto cumple los criterios.	☐ Está armado, con ajustes pendientes.	☐ Aún está en construcción.
Reflexión 5 dimensiones	☐ Respondí cada una con honestidad.	☐ Respondí algunas con detalle.	☐ Mis respuestas son muy generales.
Triángulo de pensamiento	☐ Conecté las 3 voces con mi trabajo.	☐ Conecté al menos una.	☐ No las relaciono con mi proceso.

Hoy entendí que:

Mi compromiso para la próxima sesión es:

Cita de despedida · Marco Aurelio

“El obstáculo en el camino se vuelve el camino. Lo que estorba al camino se vuelve camino.”

Cada error de hoy, bien observado, se convierte en pieza del camino que pisarás mañana.

Nombre completo:

Fecha: _____ Curso/grupo: _____

Mi firma: _____

Para la próxima sesión. Llega con tu programa de 2 sensores y la bitácora de 5 pruebas. En la sesión 5 vas a aprender a usar **actuadores** (LEDs, sonido, coreografía) para que el micro:bit no solo perciba sino también se exprese con sofisticación. Los sensores de hoy son la entrada; los actuadores de mañana son la salida creativa.